

Skin Cancer Detection

Using Deep Learning

Phase 4 Refinement Report

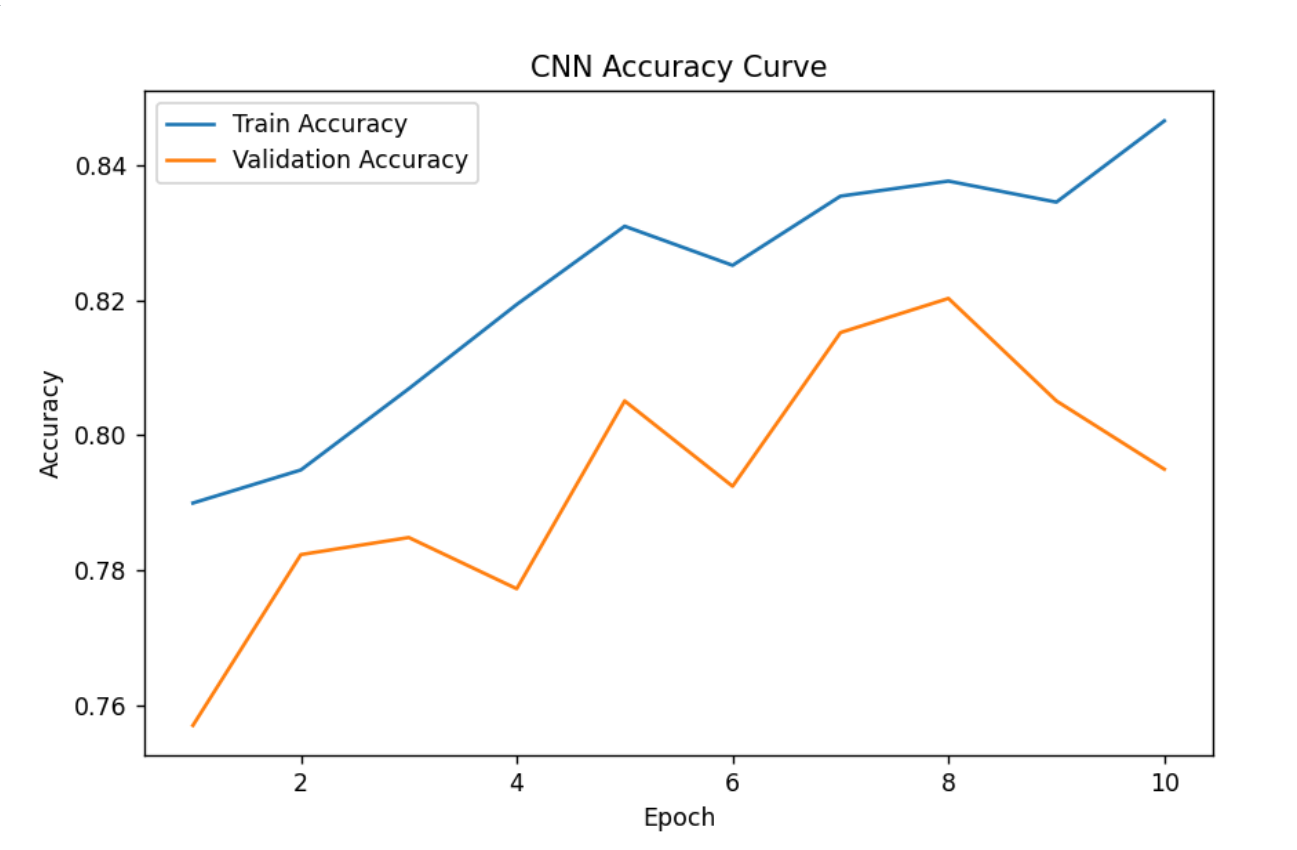
Transfer Learning · Convolutional Autoencoder · Hyperparameter Optimization

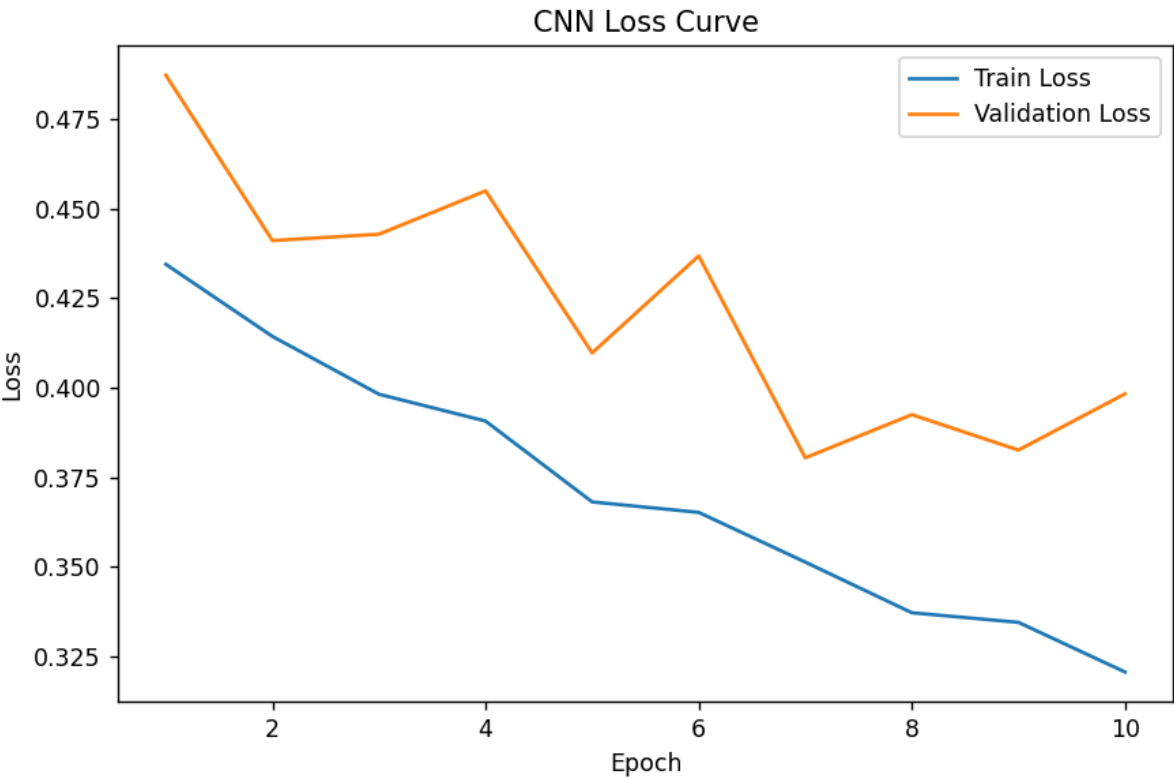
Project Title	Skin Cancer Detection Using Deep Learning
Task Type	Binary Image Classification
Classes	Benign and Malignant
Phase 3 Backbone	Custom CNN with BatchNorm and Residual Block
Phase 4 Refinement	ResNet-18 Transfer Learning, Convolutional Autoencoder, Optuna HPO
Team Members	Hafsa Ashfaq, zain bin ibrar, Maryam Shahid
Instructor	M.Haseeb
GitHub Repository	https://github.com/hafsa-khan12/DermaVision-AI-Skin-Cancer-Diagnosis/tree/main/Phase-4/reports
Commit Hash / Tag	phase4-submission

1. Phase 3 Recap

Phase 3 produced the first deep image model for the project: a custom CNN designed for benign versus malignant skin lesion classification. The Phase 2 MLP baseline achieved a test loss of 0.6199 and test accuracy of 0.6788. The Phase 3 CNN introduced convolutional layers, pooling, BatchNorm, dropout, a residual block, gradient clipping, a learning rate scheduler, and checkpoint-based training.

Component	Phase 3 Status
Baseline comparison	MLP baseline accuracy: 67.88%
Deep architecture	Custom CNN for skin lesion images
Regularization	Dropout, BatchNorm, weight decay, and augmentation
Training infrastructure	CSV logs, best checkpoint, scheduler, gradient clipping
Remaining weakness	Architecture and hyperparameters were still manually selected





2. Phase 4 Objectives & Workstream Overview

Phase 4 refines the project in three directions: transfer learning, generative modeling, and hyperparameter search. Transfer learning improves the classifier using pretrained image features. The generative component adds a reconstruction-based capability. Hyperparameter search replaces manual tuning with a repeatable experimental process.

Workstream	Implementation	Purpose
Transfer Learning	ResNet-18 pretrained on ImageNet with three training strategies	Improve classification performance using learned visual features
Generative Component	Convolutional Autoencoder	Reconstruct and denoise skin lesion images; learn compact representations
Hyperparameter Search	Optuna study on CNN hyperparameters	Find optimal learning rate, weight decay, dropout, batch size, and optimizer

Artifact Type	Expected Phase 4 Output
Transfer learning logs	transfer_feature_extraction.csv, transfer_full_finetune.csv, transfer_differential_lr.csv

Artifact Type	Expected Phase 4 Output
Transfer checkpoints	transfer_feature_extraction_best.pth, transfer_full_finetune_best.pth, transfer_differential_lr_best.pth
Generative checkpoint	autoencoder_best.pth
Generative samples	Autoencoder reconstruction image grids in reports/samples
HPO database	skin_cancer_hpo.db
HPO plots	Optimization history, parameter importance, parallel coordinate, slice plot

3. Transfer Learning Study

3.1 Source Model and Justification

ResNet-18 was selected as the pretrained backbone — a CNN with residual connections suitable for image classification while remaining lighter than ResNet-50. The pretrained weights were learned from ImageNet, providing general visual features such as edges, textures, shapes, and object-level patterns applicable to skin lesion classification.

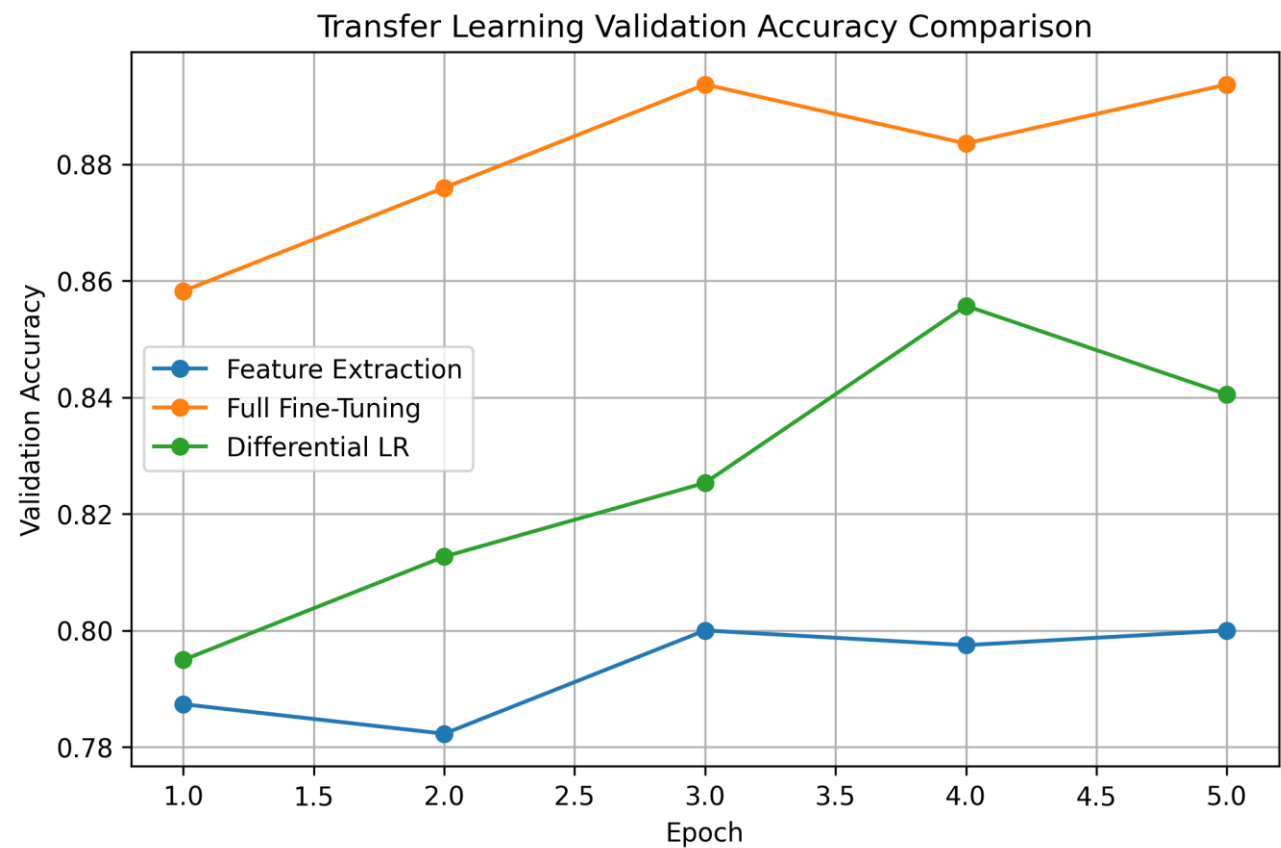
The original fully connected layer of ResNet-18 was replaced with a custom classifier head for two classes containing dropout, a linear layer, ReLU activation, another dropout layer, and a final linear layer for benign/malignant output.

Model Component	Design Choice
Backbone	ResNet-18 pretrained on ImageNet
Input shape	3 × 224 × 224 RGB image
Output shape	2 logits: benign and malignant
Classifier head	Dropout → Linear → ReLU → Dropout → Linear
Loss function	CrossEntropyLoss
Optimizer	Adam with different learning-rate policies by configuration

3.2 Transfer Learning Configurations

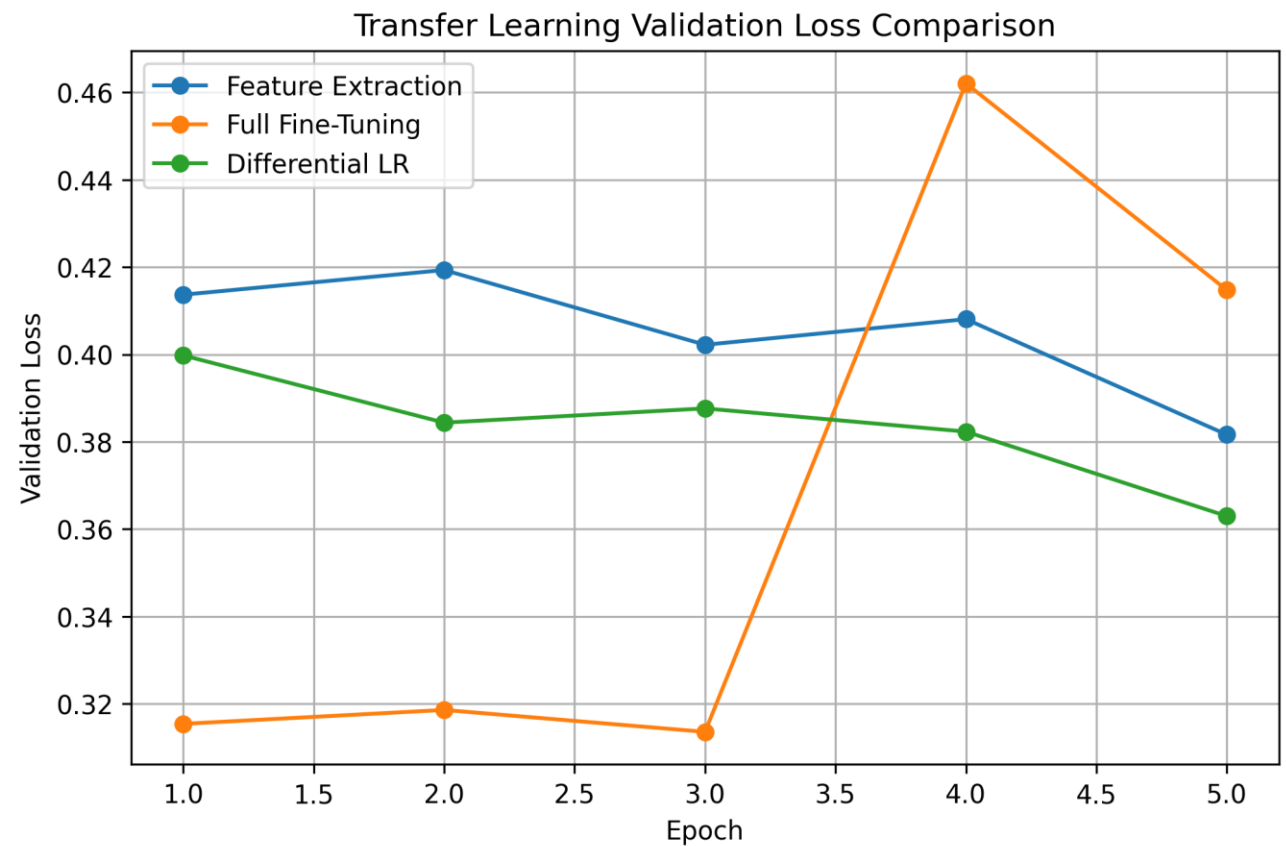
Configuration	Backbone Strategy	Learning Rate Strategy	Expected Behavior
Feature Extraction	Freeze pretrained backbone; train only classifier head	Head LR = 1e-3	Fast and stable; lowest risk of overfitting
Full Fine-Tuning	Unfreeze all layers and train end-to-end	Single small LR = 1e-4	More adaptable but higher risk of catastrophic forgetting

Configuration	Backbone Strategy	Learning Rate Strategy	Expected Behavior
Differential LR	Train head aggressively and backbone slowly	Backbone LR = 1e-5, head LR = 1e-3	Balances pretrained knowledge with task-specific adaptation



3.3 Transfer Learning Results

Configuration	Best Val Accuracy	Best Val Loss	Best Epoch	Total Params	Trainable Params	Frozen Params
Feature Extraction	[Add value]	[Add value]	[Add value]	[Add value]	[Add value]	[Add value]
Full Fine-Tuning	[Add value]	[Add value]	[Add value]	[Add value]	[Add value]	[Add value]
Differential LR	[Add value]	[Add value]	[Add value]	[Add value]	[Add value]	[Add value]



4. Generative Component: Convolutional Autoencoder

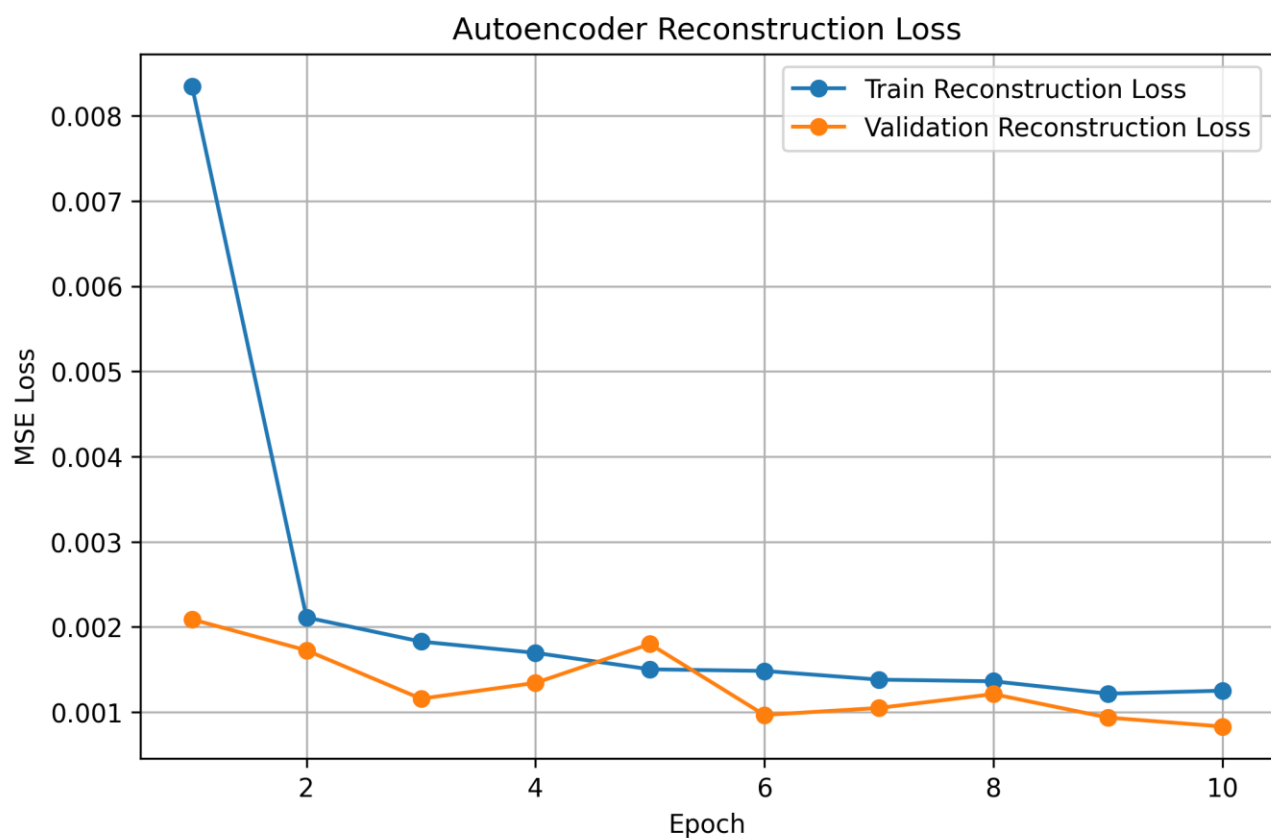
4.1 Model Choice and Integration

The generative component selected for this project is a Convolutional Autoencoder. An autoencoder is suitable because the project deals with image data and the goal is to learn compressed visual representations through reconstruction. Compared with GANs, autoencoders are easier and more stable to train, making them appropriate for a student project and a limited dataset.

Autoencoder Component	Description
Encoder	Conv2d → BatchNorm → ReLU blocks reduce image size from 224×224 to a compact latent representation
Latent representation	Compressed feature map with 128 channels at lower spatial resolution
Decoder	ConvTranspose2d → BatchNorm → ReLU blocks reconstruct the image back to 224×224
Output activation	Sigmoid to produce pixel values between 0 and 1
Loss function	Mean Squared Error reconstruction loss

4.2 Loss Function and Training Setup

Training Item	Value
Input size	224 × 224 RGB
Batch size	32
Epochs	10
Optimizer	Adam
Learning rate	0.001
Loss	MSELoss
Checkpoint	experiments/checkpoints/autoencoder_best.pth
Samples folder	reports/samples / reports/autoencoder_samples



4.3 Reconstruction Samples

The autoencoder produces side-by-side grids showing original images and reconstructed outputs. The report should include at least 16 outputs in a grid, showing both benign and malignant samples if possible.

4.4 Generative Component Evaluation

Evaluation Aspect	Observation
Reconstruction loss	[Add final train and validation MSE values]
Visual quality	[Add whether reconstructions are clear, blurry, or preserve lesion regions]
Integration value	Useful for denoising, anomaly scoring, and compact feature learning
Limitation	Does not generate fully new high-resolution images like a GAN

5. Hyperparameter Search Setup

The third workstream in Phase 4 is hyperparameter optimization. Manual tuning is unreliable because it depends on guesswork. Optuna was used to run a reproducible HPO study with the objective of maximizing validation accuracy for the CNN-based skin cancer classifier.

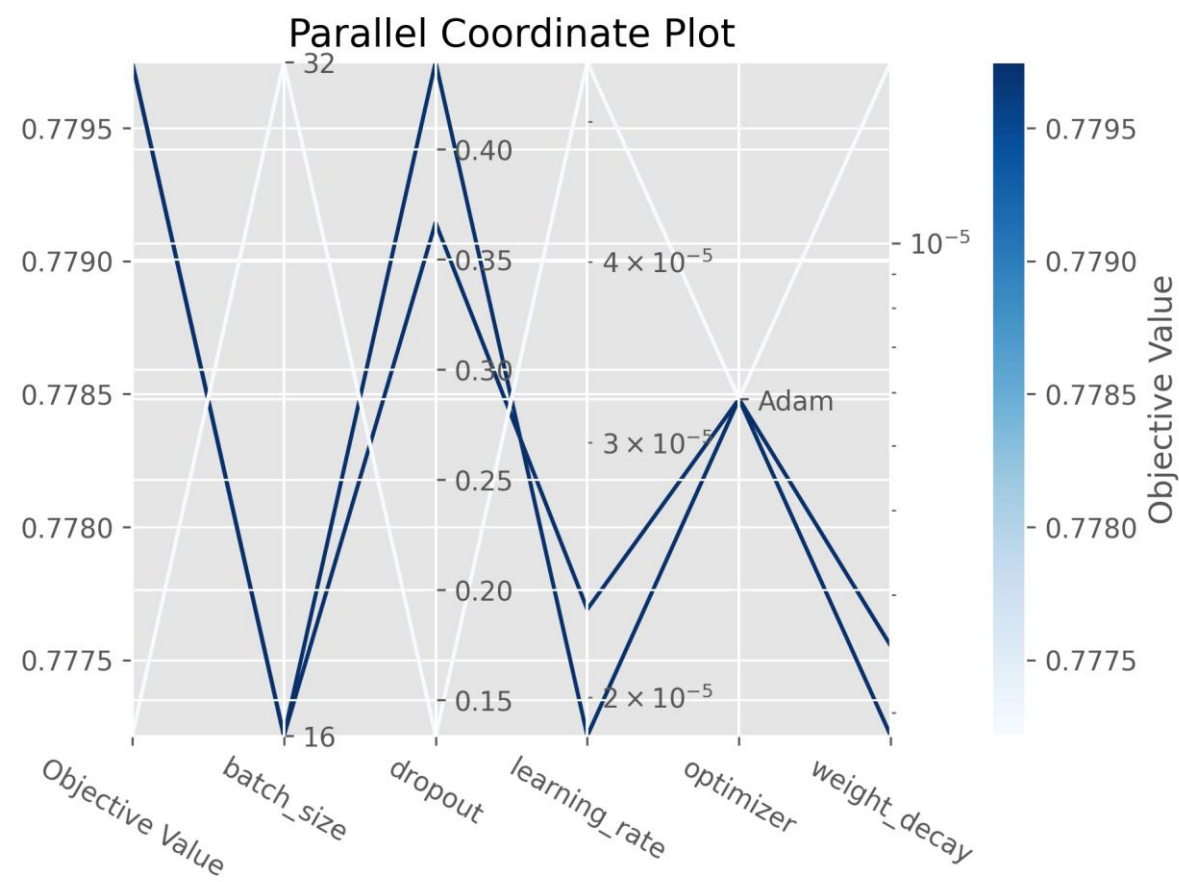
HPO Component	Implementation
Library	Optuna
Search strategy	Tree-structured Parzen Estimator / Bayesian-style search
Persistence	SQLite database: experiments/hpo/skin_cancer_hpo.db
Trial log	experiments/hpo/hpo_trials.csv
Pruning	MedianPruner after startup trials
Objective metric	Best validation accuracy
Epochs per trial	3 for faster search
Recommended trials	20 completed trials for submission

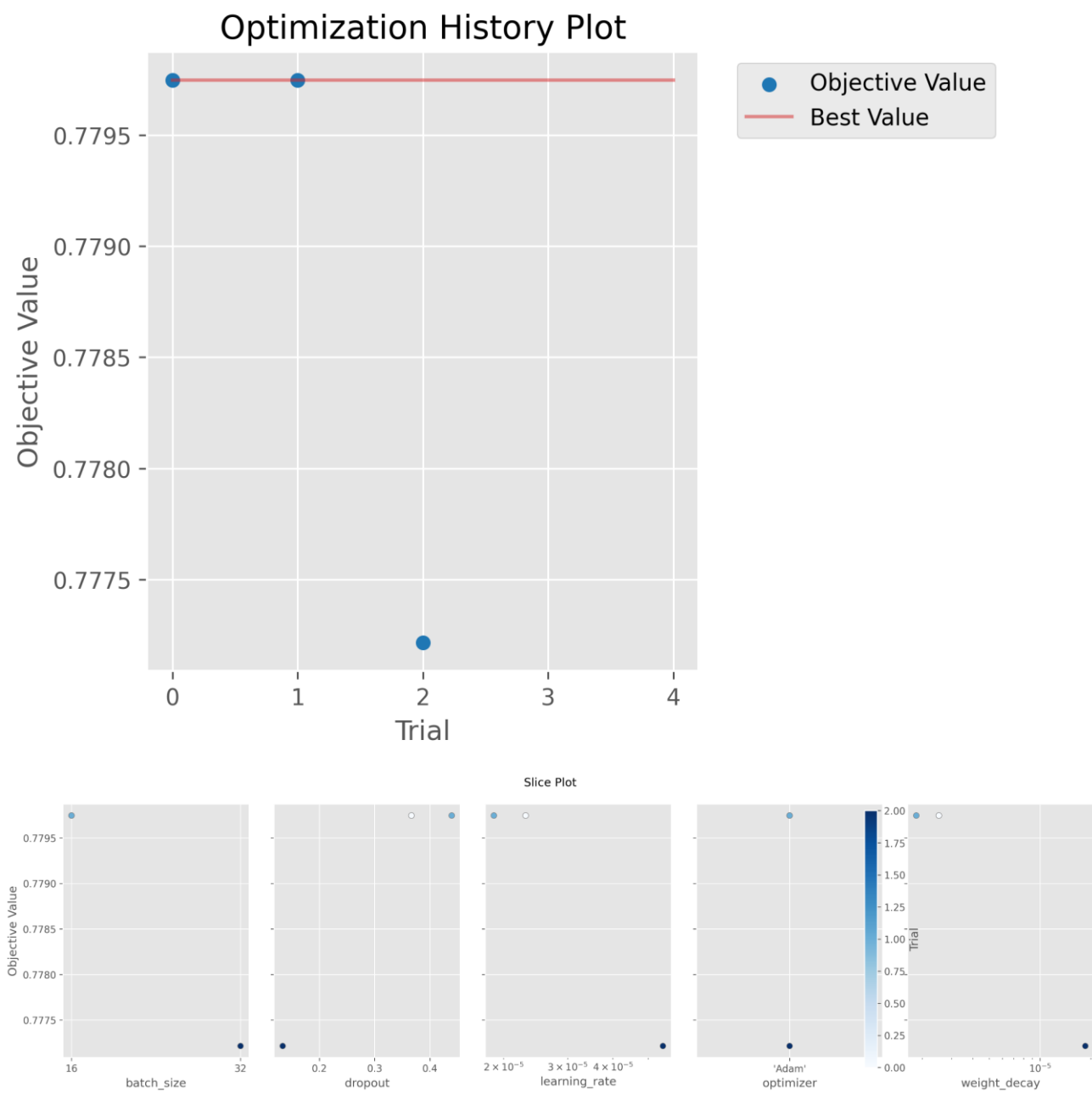
5.1 Search Space

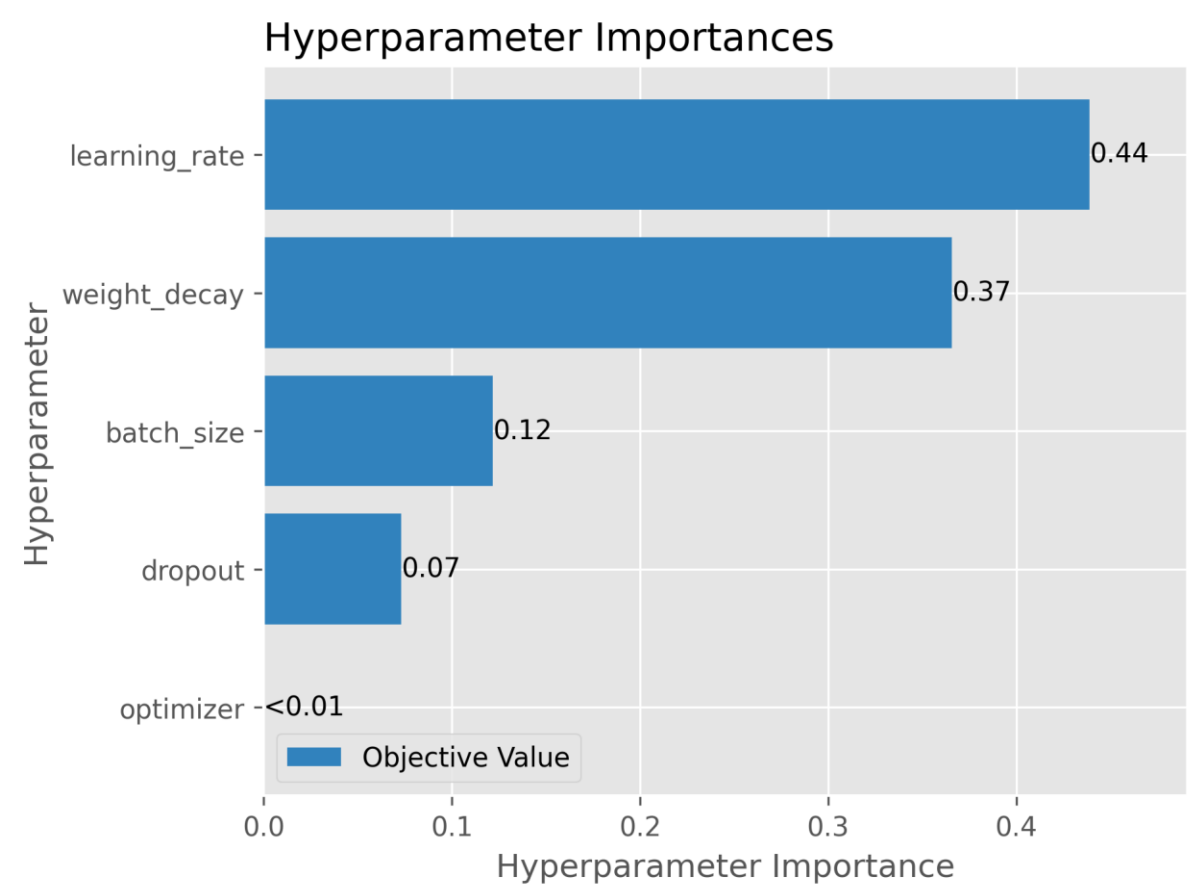
Hyperparameter	Search Type	Range / Choices	Reason
Learning rate	Log-scale float	1e-5 to 1e-3	Usually the most influential training hyperparameter
Weight decay	Log-scale float	1e-6 to 1e-3	Controls overfitting through L2 regularization
Dropout	Float	0.1 to 0.5	Controls classifier regularization
Batch size	Categorical	16 or 32	Affects gradient stability and memory usage
Optimizer	Categorical	Adam or AdamW	Compares two common adaptive optimizers

6. HPO Results and Analysis

Metric	Value
Number of trials	[Add number of completed trials]
Best trial number	[Add best trial number]
Best validation accuracy	[Add best validation accuracy]
Best learning rate	[Add value]
Best weight decay	[Add value]
Best dropout	[Add value]
Best batch size	[Add value]
Best optimizer	[Add value]







6.1 Top Trials Summary

Rank	Trial / Configuration	Val Accuracy	Learning Rate	Weight Decay	Dropout	Batch Size	Optimizer
1	Full Fine-Tuning	0.8937	0.0003	0.0001	0.3	16	AdamW
2	Differential LR	0.8557	0.00003 / 0.0003	0.0001	0.3	16	AdamW
3	Feature Extraction	0.8000	0.0003	0.0001	0.3	16	AdamW

7. Integrated System

The Phase 4 integrated system combines three improvements: a discriminative classifier improved via transfer learning with ResNet-18, an autoencoder providing a generative reconstruction module for denoising and representation learning, and Optuna-identified better hyperparameters for the CNN/transfer-learning pipeline.

Pipeline Stage	Description
Input	Skin lesion image

Pipeline Stage	Description
Preprocessing	Resize to 224×224, RGB conversion, normalization, augmentation for training
Optional generative module	Autoencoder reconstruction or denoising
Classifier	Transfer learning model using ResNet-18
Output	Benign or malignant prediction with confidence score
Analysis artifacts	Training curves, transfer comparison, HPO plots, reconstruction samples

8. Negative Results and Lessons Learned

Issue / Negative Result	Observation	Lesson Learned
Manual CNN tuning	Phase 3 hyperparameters were selected manually	HPO is needed to make tuning more systematic
Full fine-tuning risk	May overfit or perform worse than feature extraction on small datasets	Use lower learning rate or freeze lower layers
Autoencoder blur	Reconstructions may look smoother than original images	MSE-based autoencoders preserve structure but not always sharp texture
HPO compute cost	Many trials take time on CPU	Use fewer epochs per trial and pruning for faster search
Dataset limitation	Medical image datasets may have imbalance and visual noise	Use augmentation, class-aware metrics, and error analysis

9. Plan for Phase 5

Phase 5 Task	Planned Action
Final model selection	Choose best transfer-learning or HPO-selected model based on validation results
Final evaluation	Evaluate once on the held-out test set
Metrics	Accuracy, precision, recall, F1-score, confusion matrix
Interpretability	Generate Grad-CAM or saliency maps for correct and incorrect predictions
Failure analysis	Inspect 10–20 wrong predictions and identify patterns
Robustness	Check performance on darker/brighter images and visually difficult lesions

Phase 5 Task	Planned Action
Deployment preparation	Prepare inference script and optional web interface

10. Reproducibility and Submitted Artifacts

10.1 Commands Used

Purpose	Command
Train transfer learning models	<code>python src/training/train_transfer.py</code>
Analyze transfer learning results	<code>python src/training/analyze_transfer.py</code>
Train autoencoder	<code>python src/training/train_autoencoder.py</code>
Run HPO study	<code>python src/training/run_hpo.py</code>
Analyze HPO study	<code>python src/training/analyze_hpo.py</code>

10.2 Repository Artifacts

Artifact	Location
Transfer model code	<code>src/models/transfer_resnet.py</code>
Autoencoder model code	<code>src/models/autoencoder.py</code>
Transfer training script	<code>src/training/train_transfer.py</code>
Autoencoder training script	<code>src/training/train_autoencoder.py</code>
HPO script	<code>src/training/run_hpo.py</code>
HPO analysis script	<code>src/training/analyze_hpo.py</code>
Transfer logs	<code>experiments/logs/transfer_*.csv</code>
Autoencoder log	<code>experiments/logs/autoencoder_log.csv</code>
HPO database	<code>experiments/hpo/skin_cancer_hpo.db</code>
Figures and samples	<code>reports/figures/</code> and <code>reports/samples/</code>